Blended GEV: a tutorial using R-INLA

Version 1.2

Daniela Castro-Camilo • daniela.castrocamilo@glasgow.ac.uk

A reparametrisation of the GEV

The blended Generalised extreme value (bGEV) model is an alternative to the usual GEV distribution when the tail parameter ξ is positive, and it is designed to tackle the *artificial boundary restrictions* inherited by the GEV. By artificial boundary restrictions, we mean the following: in practical applications, we assume that the GEV is a reasonable approximation for the distribution of maxima over blocks, and we fit it accordingly. This implies that GEV properties, such as finite lower endpoint in the case $\xi > 0$, are inherited by the original maxima distribution, which might not be bounded-supported. This is particularly problematic in the presence of covariates.

Before defining the bGEV distribution, we need to introduce an alternative parametrisation for the GEV. The GEV distribution parametrised in terms of the **location** $q_\alpha \in \mathbb{R}$, the **spread** $s_\beta \in (0, \infty)$ and the **tail** parameter $\xi \in \mathbb{R}$ has the form

$$F(y \mid q_\alpha, s_\beta, \xi) = \exp\left\{-\left[\left(\frac{y - q_\alpha}{s_\beta(\ell_{1-\beta/2,\xi} - \ell_{\beta/2,\xi})^{-1}} + \ell_{a,\xi}\right)\right]_+^{-1/\xi}\right\},$$

where $a_+ = \max(a, 0)$ and for any $a > 0$, $\ell_{a,\xi} = \{-\log a\}^{-\xi}$. Note that the case $\xi = 0$ simplifies to

$$F(y \mid q_\alpha, s_\beta) = \exp\left\{-\exp\left[-\left(\frac{y - q_\alpha}{s_\beta(\ell_{1-\beta/2} - \ell_{\beta/2})^{-1}} - \ell_\alpha\right)\right]\right\},$$

with $\ell_a = \log\{-\log a\}$. There is a one-to-one mapping between (q_α, s_β, ξ) and the usual location-scale-shape GEV parameters, (μ, σ, ξ) (see, e.g., Coles, 2001, Chapter 3). For the case $\xi \neq 0$, the mapping is given by

$$\mu = q_\alpha - \frac{s_\beta(\ell_{\alpha,\xi} - 1)}{\xi(\ell_{1-\beta/2,\xi} - \ell_{\beta/2,\xi})}, \qquad \sigma = \frac{s_\beta}{(\ell_{1-\beta/2,\xi} - \ell_{\beta/2,\xi})}, \qquad \xi = \xi.$$

The case $\xi = 0$ is interpreted as the limit when $\xi \to 0$, i.e.,

$$\mu = q_\alpha + \frac{s_\beta \ell_\alpha}{\ell_{\beta/2} - \ell_{1-/\beta/2}}, \qquad \sigma = \frac{s_\beta}{\ell_{\beta/2} - \ell_{1-/\beta/2}}.$$

This reparametrisation is proposed to provide a more meaningful interpretation of the parameters. In statistics, the location-scale parametrisation is quite popular as it relates to the mean and the standard deviation of the distribution. In skewed distributions such as the GEV, the mean is no longer a reasonable proxy for the location of the distribution. Moreover, the mean and variance of the GEV are only defined when $\xi < 1$ and $\xi < 0.5$, respectively. This effect of the tail parameter over the mean and variance does not allow us to interpret the location-scale (and tail) GEV parametrisation as we do in other models. This problem is particularly troublesome for the case where the parameters vary according to a set of covariates. Assigning sensible priors to the GEV distribution with the usual parametrisation is also tricky when the mean and variance are not defined.

The blended GEV model

The bGEV distribution is defined as

$$\mathrm{H}(x \mid \theta) = \mathrm{F}(x \mid q_{\alpha_\mathrm{F}}, s_{\beta_\mathrm{F}}, \xi)^{p(x|s_{p_1}, s_{p_2}, a, b)} \mathrm{G}(x \mid q_{\alpha_\mathrm{G}}, s_{\beta_\mathrm{G}})^{1-p(x|s_{p_1}, s_{p_2}, a, b)}, \tag{1}$$

where $\theta = (q_{\alpha_\mathrm{F}}, s_{\beta_\mathrm{F}}, \xi, q_{\alpha_\mathrm{G}}, s_{\beta_\mathrm{G}}, s_{p_1}, s_{p_2}, a, b)$, F is the Frechét (or type II GEV) distribution with location parameter q_{α_F}, spread parameter s_{β_F}, and **tail** parameter ξ, and G is the Gumbel (or type I GEV) distribution with location parameter q_{α_G} and spread parameter s_{β_G}. The function p is a weight function defined as the cumulative distribution function of a Beta distribution with shape parameters $s_{p_1} > 1$ and $s_{p_2} > 1$, evaluated in the point $(x - a)/(b - a)$, i.e.,

$$p(x \mid s_{p_1}, s_{p_2}, a, b) = \Pr\left(Y \leq \frac{x - a}{b - a} \mid s_{p_1}, s_{p_2}\right), \tag{2}$$

where Y follows a Beta distribution with shape parameters $s_{p_1} > 1$ and $s_{p_2} > 1$. The Beta weight controls the way the distributions F (Frechét) and G (Gumbel) influence the model H. The lower and upper bounds of the weight function (a and b, respectively) define the *mixing area*, i.e., where F and G are merged (see Figure 1). Here, we choose them as quantiles of the Frechét distribution, i.e., $a = F^{-1}(p_a)$ and $b = F^{-1}(p_b)$, with $0 < p_a, p_b < 1$. Below, p_a and p_b will be refered as the *mixing area quantiles*. The current INLA implementation assumes that $s_{p_1} = s_{p_2} = 5$.

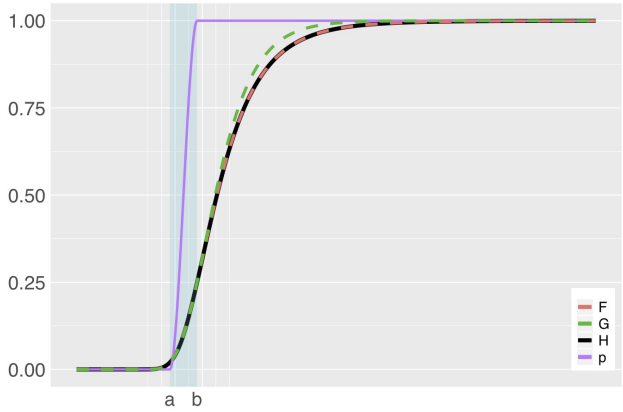


Figure 1: bGEV distribution (H, black) constructed from distributions F (Frechét, red), G (Gumbel, green) and Beta weight function p (purple). The shaded area is the mixing area, where F and G are merged.

In the following sections, we will learn how to fit the bGEV using R-INLA using three simulated examples with increasing level of complexity.

Simulated example 1

To get familiar with the bGEV R-INLA implementation, we consider a simple model where the linear predictor is linked to the $\alpha-$quantile, q_α. The model we want to fit is

$$q_\alpha = \eta(\mathtt{x}) = 1 + 0.4\mathtt{x} \tag{3}$$

Data simulation

We start by generating $n = 1000$ samples from (3)

```
n = 1000
x = rnorm(n, sd=0.5) # we generate values for x from a N(0,0.5^2) dist.
eta.x = 1 + 0.4*x
```

The spread and tail parameters are assumed to be covariate-free and unknown and are treated as hyperparameters within the INLA framework. We assume that the true spread and tail parameters are 0.3 and 0.1, respectively.

```
spread = 0.3
tail = 0.1
```

To generate the GEV samples, we need to define the probabilities α and β that define the location (q_α) and spread (s_β) parameters, respectively. In our case, they will be fixed to $\alpha = 0.5$ (the median) and $\beta = 0.25$:

```
p.alpha = 0.5
p.beta = 0.25
```

Now we are ready to generate the samples. We use the function `giveme.gev.par` (See Section 5) to obtain the usual GEV parameters (μ, σ, ξ) and plug them into the `evd::rgev` function to generate the samples

```
par = giveme.gev.par(q = eta.x, sbeta = spread, alpha = p.alpha, beta = p.beta,
                     xi = tail)
y = numeric(n)
for(i in 1:n)
  y[i] = rgev(1, loc = par$mu[i], scale = par$sigma, shape = par$xi)
```

Prior specification

The default prior distribution for the spread s_β is a Gamma with shape and rate parameters equal to 3 (note that a log-scale is used below). For the tail parameter we consider a PC prior approach with parameters $\lambda = 7$, `low` $= 0$ and `high` $= 0.5$. For more details on the PC prior for the tail GEV parameter see Section 4.2.

For the sake of illustration, we here define the priors for all the parameters involved. The priors for the spread is

```
hyper.spread = list(initial = 1,
                    fixed=FALSE,
                    prior = "loggamma",
                    param = c(3, 3))
```

The prior for the tail parameter requires a bit more explanation. For computational reasons, R-INLA uses an internal parametrisation of the tail parameter, which is unbounded. The hyperparameter specification is defined for this internal parameter instead of the usual one, so in order to know how to specify a prior for the tail parameter, we need to understand how both parametrisations are connected.

The map $\phi : [0, \infty) \to \mathbb{R}$ specifies the link between the usual and the internal parametrisations and it is defined as

$$\phi(\xi_{\text{int}}) = \texttt{low} + (\texttt{high} - \texttt{low})\frac{\exp(\xi_{\text{int}})}{1 + \exp(\xi_{\text{int}})} := \xi,$$

where $\xi \in [0, \infty)$ is the usual tail parameter and ξ_{int} refers to the unbounded internal tail parameter. The interval $(\texttt{low}, \texttt{high})$ constraints the possible values for ξ. The map and its inverse are defined in the function `map.tail` (See Section 5).

In the internal parametrisation, the default initial value is -4 with $(\texttt{low}, \texttt{high}) = (0, 0.5)$, which correspond to $\xi \approx 0.04$. If we want to provide an initial value of $\xi = 0.1$, then we can do

```
tail.interval = c(0, 0.5)
tail.intern = map.tail(tail, tail.interval, inverse=TRUE)
```

We kept $(\texttt{low}, \texttt{high}) = (0, 0.5)$ to ensure the existence of second moments[1].

The prior for the tail parameter is defined as

```
hyper.tail = list(initial = tail.intern,
                  prior = "pc.gevtail",
                  param = c(7, tail.interval),
                  fixed= FALSE)
```

Note: if we have reasons to believe that $\xi = 0$ (Gumbel case), then a good initial value is $-\infty$ (in the internal parametrisation). We can generalise the prior specification for the tail to allow $\xi > 0$ or $\xi = 0$ as follows

```
hyper.tail = list(initial = if (tail == 0.0) -Inf else tail.intern,
                  prior = "pc.gevtail",
                  param = c(7, tail.interval),
                  fixed= if (tail == 0.0) TRUE else FALSE)
```

Therefore, the (default) hyperparameter specification for the bGEV model is

```
hyper.bgev = list(spread = hyper.spread,
                  tail = hyper.tail)
```

Control variables

As part of the `control.family` argument in R-INLA, the argument `control.bgev` allows us to include additional bGEV parameters. Specifically, the probabilities α and β, the mixing area quantiles p_a and p_b, and the Beta weight function parameters, which we know are equal and fixed to 5 (for now, this cannot be changed).

```
control.bgev = list(q.location = p.alpha,
                    q.spread = p.beta,
                    # quantile levels for the mixing part
                    q.mix= c(0.05, 0.20),
                    # the Beta(s1, s2) mixing distribution parameters.
                    # Hard-coded for the moment: s1=s2=5
                    beta.ab = 5)
```

[1] Recall that the variance for the GEV is defined for $\xi < 0.5$.

INLA fit

The INLA formula for the bGEV model uses the function `inla.mdata` to allow the inclusion of simple linear models in the spread and the tail parameters (for details on the `inla.mdata` function see Section 4.3). A null matrix can be used to indicate that these parameters are covariate-free (as it is the case in this example).

```
null.matrix = matrix(nrow = n, ncol= 0)
spread.x = null.matrix
tail.x = null.matrix
```

Note: matrices for the spread and tail covariates (here spread.x and tail.x) should always be defined and passed to the R-INLA formula, even if they are empty.

Then, the INLA data and formula can be defined as

```
data.bgev = data.frame(y = y, intercept = 1, x = x, spread.x = spread.x, tail.x = tail.x)
formula = inla.mdata(y, spread.x, tail.x) ~ -1 + intercept + x
```

Note: `data.bgev` only contains three columns, as null matrices cannot be passed to data frames in R. As `spread.x` and `tail.x` are not defined in `data.bgev`, INLA will search for these variables in the global R environment. Therefore, `spread.x` and `tail.x` should be defined as null matrices in the global environment (as we do here). We write `data.bgev` this way for consistency with the following examples.

Finally, we fit the model

```
r1 = inla(formula,
          family = "bgev",
          data = data.bgev,
          control.family = list(hyper = hyper.bgev,
                                control.bgev = control.bgev),
          control.predictor = list(compute = TRUE),
          control.fixed = list(prec=1000),
          control.compute = list(cpo = TRUE),
          control.inla = list(int.strategy = "eb"),
          verbose=FALSE, safe=TRUE)
```

A summary of the fitted fixed effects and hyperparameters can be obtained as follows

```
round(r1$summary.fixed,4)
```

```
##             mean     sd 0.025quant 0.5quant 0.975quant  mode kld
## intercept 0.987 0.0031      0.981    0.987      0.993 0.987   0
## x         0.380 0.0058      0.368    0.380      0.391 0.380   0
```

```
round(r1$summary.hyperpar,4)
```

```
##                               mean     sd 0.025quant 0.5quant
## spread for BGEV observations 0.284 0.0078     0.2687    0.284
## tail for BGEV observations   0.108 0.0243     0.0652    0.107
##                              0.975quant  mode
## spread for BGEV observations      0.299 0.283
## tail for BGEV observations        0.160 0.105
```

Simulated example 2

Although R-INLA does not allow more than one linear predictor, the bGEV implementation does allow for simpler regression models on the spread and tail parameters. We then extend model (3) as follows

$$\begin{aligned} q_\alpha &= \eta(\mathtt{x}_1) = 1 + 0.4\mathtt{x}_1 \\ s_\beta &= \exp(0.1 + 0.3\mathtt{x}_2) \\ \xi &= 0.1 + 0.2\mathtt{x}_3 \end{aligned} \tag{4}$$

Data simulation

As before, we start by generating $n = 1000$ samples from (4)

```
n = 1000
x1 = rnorm(n)
eta.x = 1 + 0.4*x1
```

The spread and tail parameter can be simulated as follows

```
x2 = rnorm(n, sd= 0.2)
s.x = exp(0.1 + 0.3*x2)
x3 = runif(n,-0.25,1)
t.x = 0.1 + 0.2*x3
tail.intern = map.tail(t.x, tail.interval, inverse=TRUE) # internal xi
```

Note that since $-0.5 < \mathtt{x}_3 < 2$, we have that $0 < \xi < 0.5$. As before, we use the function `giveme.gev.par` to obtain the usual GEV parameters and plug them into the `evd::rgev` function to generate the samples (note that α and β are the same as before)

```
par = giveme.gev.par(q = eta.x, sbeta = s.x, alpha = p.alpha, beta = p.beta,
                     xi = t.x)
y = numeric(n)
for(i in 1:n)
  y[i] = rgev(1, loc = par$mu[i], scale = par$sigma[i], shape = par$xi[i])
```

Prior specification

Additional to what was discussed in Section 3.2, we can also adjust the priors for the regression coefficient of the covariates for the spread and tail parameters, which we will call β_1 and β_2. Within INLA, β_1 and β_2 are treated as hyperparameters, with default prior given by a zero-mean Gaussian distribution with precision equal to 300, as specified below.

```
hyper.beta1 = hyper.beta2 = list(prior = "normal",
                                 param = c(0, 300),
                                 initial = 0)
```

In this case, the (default) hyperparameter specification for the bGEV model is

```
hyper.bgev = list(spread = hyper.spread,
                  tail = hyper.tail,
                  beta1 = hyper.beta1,
                  beta2 = hyper.beta2)
```

INLA fit

As mentioned in Section 3.4, we can use the `inla.mdata` function to define linear models for the spread and the tail parameters.

```
spread.x = x2
tail.x = x3
formula = inla.mdata(y, spread.x, tail.x) ~ -1 + intercept + x
```

Then, the INLA data can be defined as

```
data.bgev = data.frame(y = y, intercept = 1, x = x, spread.x = spread.x, tail.x = tail.x)
```

We fit the model using the same priors and mixing area quantiles (p_a, p_b) as before.

```
r2 = inla(formula,
          family = "bgev",
          data = data.bgev,
          control.family = list(hyper = hyper.bgev,
                                control.bgev = control.bgev),
          control.predictor = list(compute = TRUE),
          control.fixed = list(prec=1000),
          control.compute = list(cpo = TRUE),
          control.inla = list(int.strategy = "eb"),
          verbose=FALSE, safe=TRUE)
```

A summary of the fitted fixed effects and hyperparameters can be obtained as follows

```
round(r2$summary.fixed,4)
```

```
##               mean    sd 0.025quant 0.5quant 0.975quant    mode kld
## intercept  0.7225 0.018     0.6873   0.7225     0.7578  0.7225   0
## x         -0.0202 0.026    -0.0711  -0.0202     0.0307 -0.0202   0
```

```
round(r2$summary.hyperpar,4)
```

```
##                                       mean     sd 0.025quant
## spread for BGEV observations        1.4978 0.0365     1.4276
## tail for BGEV observations          0.0275 0.0137     0.0072
## beta1 (spread) for BGEV observations 0.0711 0.0531    -0.0332
## beta2 (tail) for BGEV observations   0.0027 0.0577    -0.1100
##                                     0.5quant 0.975quant   mode
## spread for BGEV observations          1.4972     1.5714 1.4958
## tail for BGEV observations            0.0254     0.0588 0.0198
## beta1 (spread) for BGEV observations  0.0710     0.1760 0.0705
## beta2 (tail) for BGEV observations    0.0024     0.1174 0.0010
```

Simulated example 3

The linear predictor can include more complicate structures defined as functions that depend on a set of covariates. By varying the form of these functions, we can accommodate a wide range of models, from standard and hierarchical regression to spatial and spatio-temporal models (Rue et al., 2009). Here we extend the model in (4) by assuming the following structure

$$\begin{aligned} q_\alpha &= \eta(\mathtt{x}_1, \mathtt{z}_1, \mathtt{z}_2) = 1 + 0.4\mathtt{x}_1 + f_1(\mathtt{z}_1) + f_2(\mathtt{z}_2) \\ s_\beta &= \exp(0.1 + 0.3\mathtt{x}_2 + \mathtt{x}_4) \\ \xi &= 0.1 + 0.2\mathtt{x}_3 \end{aligned} \tag{5}$$

where f_1 is a random walk of order 1 and f_2 is an autoregressive process of order 2. Note that we also extended the linear model for the spread parameter.

Data simulation

There are many ways we can simulate dara to fit (5). One alternative is

```
n = 1000
x = rnorm(n)
z1 = seq(0, 6, length.out = n)
z2 = 1:n
p = 2 # AR order
pacf = runif(p)
phi = inla.ar.pacf2phi(pacf)
eta.x = 1 + 0.4*x + sin(z1) + c(scale(arima.sim(n, model = list(ar = phi))))
```

The spread and tail parameter are simulated as before:

```
x2 = rnorm(n, sd = 0.2)
x4 = rnorm(n, sd = 0.2)
s.x = exp(0.1 + 0.3*x2 + x4)
x3 = runif(n,-0.2, 0.2)
t.x = 0.1 + 0.2*x3
tail.intern = map.tail(t.x, tail.interval, inverse=TRUE) # internal xi
```

The data are simulated as

```
par = giveme.gev.par(q = eta.x, sbeta = s.x, alpha = p.alpha, beta = p.beta,
                     xi = t.x)
y = numeric(n)
for(i in 1:n)
  y[i] = rgev(1, loc = par$mu[i], scale = par$sigma[i], shape = par$xi[i])
```

INLA fit

We have two covariates for the spread parameter and one for the tail, and we can have priors for each of their coefficients. Below, `hyper.beta1` and `hyper.beta2` are the priors for the coefficients of the covariates in the spread parameters, while `hyper.beta3` is the prior for the coefficient of the covariate in the tail parameter.

```
hyper.beta1 = hyper.beta2 = hyper.beta3 = list(prior = "normal",
                                               param = c(0, 300),
                                               initial = 0)
hyper.bgev = list(spread = hyper.spread,
                  tail = hyper.tail,
                  beta1 = hyper.beta1,
                  beta2 = hyper.beta2,
                  beta3 = hyper.beta3)
```

Using the same mixing area quantiles (q_a, q_b) as before, we only need to specify the INLA formula, the new data, and run the model.

```
spread.x = x2
spread.xx = x4
tail.x = x3
# With this change of variable it is easier to keep track of the effect
# of the covariates in each parameter, but it is not needed.
formula = inla.mdata(y, cbind(spread.x, spread.xx), tail.x) ~ -1 + intercept + x +
    f(z1, model = "rw1", scale.model=TRUE, constr=TRUE,
        hyper = list(prec = list(prior = "pc.prec",
        param = c(0.1, 0.01)))) +
    f(z2, model = 'ar', order = 1,
           hyper=list(prec=list(prior="pc.prec", constr=TRUE,
                                param=c(0.1,0.01)),
                      pacf1=list(param=c(0.5,0.8),
                      pacf2=list(param=c(0.5,0.8)))))
data.bgev = data.frame(y = y, intercept = 1, x = x, z1 = z1, z2 = z2,
                       spread.x = spread.x, spread.xx = spread.xx, tail.x = tail.x)
r3 = inla(formula,
         family = "bgev",
         data = data.bgev,
         control.family = list(hyper = hyper.bgev,
                               control.bgev = control.bgev),
         control.predictor = list(compute = TRUE),
         control.fixed = list(prec=1000,prec.intercept=1000),
         control.compute = list(cpo = TRUE),
         control.inla = list(int.strategy = "eb",
                             cmin=0,
                             b.strategy="keep"),
         verbose=FALSE, safe=TRUE)
```

A summary of the fitted effects and hyperparameters can be obtained as follows

```
round(r3$summary.fixed,4)
```

```
##             mean     sd 0.025quant 0.5quant 0.975quant  mode kld
## intercept 0.032 0.0311    -0.0288    0.032     0.0929 0.032   0
## x         0.221 0.0228     0.1765    0.221     0.2659 0.221   0
```

```
round(r3$summary.hyperpar,4)
```

```
##                                          mean       sd 0.025quant
```

```
## spread for BGEV observations            2.1819 9.26e-02     1.9994
## tail for BGEV observations              0.0001 1.00e-04     0.0000
## beta1 (spread) for BGEV observations    0.0252 5.43e-02    -0.0836
## beta2 (spread) for BGEV observations    0.0145 5.42e-02    -0.0937
## beta3 (tail) for BGEV observations     -0.0001 5.78e-02    -0.1163
## Precision for z1                     4485.5563 9.87e+03   101.0990
## Precision for z2                        0.9837 1.31e-01     0.7549
## PACF1 for z2                            0.9255 1.42e-02     0.8946
##                                     0.5quant 0.975quant     mode
## spread for BGEV observations         2.18e+00   2.36e+00   2.1869
## tail for BGEV observations           0.00e+00   3.00e-04   0.0000
## beta1 (spread) for BGEV observations 2.58e-02   1.30e-01   0.0286
## beta2 (spread) for BGEV observations 1.50e-02   1.20e-01   0.0171
## beta3 (tail) for BGEV observations   8.00e-04   1.11e-01   0.0044
## Precision for z1                     1.80e+03   2.57e+04 213.3670
## Precision for z2                     9.74e-01   1.27e+00   0.9534
## PACF1 for z2                         9.27e-01   9.50e-01   0.9285
```

Notes

Linear predictor for the bGEV model

The linear predictor can take any combinations of the latent structures currently implemented. To see a list of the available latent models, we can do

```
library(INLA)
inla.list.models('latent')
```

PC prior for the tail parameter

Although non-informative priors are a common choice when little expert knowledge is available, the PC prior approach allows us to select moderately informative prior distributions in a *reasonable* way. This procedure penalises excessively complex models at a constant rate by putting an exponential prior on a distance (specifically, the Kullback-Leibler distance, or KLD) to a simpler baseline model.

The PC prior for the tail GEV parameter is defined in terms of the KLD $\xi^2/(1-\xi)$ for $0 \leq \xi < 1$. In R-INLA, the prior is specified as

```
hyper = list(<theta> = list(prior="pc.gevtail", param=c(<lambda>, <low>, <high>)))
```

where

- `<lambda>`: the constant penalisation rate
- `<low>,<high>`: interval to restrict possible values for the tail parameter. The default is [0,0.5].

For more details on the PC prior for the tail GEV parameter see `inla.doc("pc.gevtail")`.

About `inla.mdata`

`inla.mdata(y, x1, x2,...)` is a matrix where each row are replicates, and responses that are `NA`s are ignored. The function `inla.mdata` accept covariates that are one or many vectors, matrices or data frames. If we pass m covariates $\texttt{x}_1, \ldots \texttt{x}_m$, then each row $(\texttt{x}_{i1}, \texttt{x}_{i2}, \ldots, \texttt{x}_{im})$ defines the covariates used for the i-th row of `y`.

Tools

The function `giveme.gev.par` computes the usual GEV parameters (μ, σ, ξ) given the bGEV parameters (q, s_β, ξ). Note that in both parametrisations, the tail parameter is the same.

```
library(evd)
giveme.gev.par = function(q, sbeta, alpha, beta, xi)
{
  .mu = function(q, sbeta, alpha, beta, xi) {
    a = -log(1-beta/2)
    b = -log(beta/2)
    c = -log(alpha)
    if (all(xi > 0.0)) {
      tmp0 = (c^(-xi) - 1)/xi
      tmp1 = a^(-xi)
      tmp2 = b^(-xi)
      dbeta = (tmp1 - tmp2)/xi
      return(q - (sbeta/dbeta) * tmp0)
    } else if (all(xi == 0.0)) {
      dbeta = log(b) - log(a)
      tmp0 = log(c)
      return(q + (sbeta/dbeta) * tmp0)
    } else {
      stop("mixed case not implemented")
    }
  }

  .sigma = function(q, sbeta, alpha, beta, xi) {
    a = -log(1-beta/2)
    b = -log(beta/2)
    if (all(xi > 0.0)) {
      tmp1 = a^(-xi)
      tmp2 = b^(-xi)
      dbeta = (tmp1 - tmp2)/xi
      return(sbeta/dbeta)
    } else if (all(xi == 0.0)) {
      dbeta = log(b) - log(a)
      return(sbeta/dbeta)
    } else {
      stop("mixed case not implemented")
    }
  }

  return(list(mu = .mu(q, sbeta, alpha, beta, xi),
              sigma = .sigma(q, sbeta, alpha, beta, xi),
              xi = xi))
}
```

The function `map.tail` specifies the link between the internal and usual parametrisations. In the code below, `interval` constraints the possible values for the tail parameter, while `inverse` is a boolean variable indicating whether ϕ or ϕ^{-1} should be computed.

```
map.tail = function(x, interval, inverse = FALSE) {
  if (!inverse) {
    return (interval[1] + (interval[2] - interval[1]) * exp(x)/(1.0 + exp(x)))
  } else {
    return (log((x-interval[1])/(interval[2]-x)))
  }
}
```

References

1. Coles, S. (2001). *An introduction to statistical modeling of extreme values* (Vol. 208, p. 208). London: Springer.

2. Rue, H., Martino, S., & Chopin, N. (2009). Approximate Bayesian inference for latent Gaussian models by using integrated nested Laplace approximations. *Journal of the royal statistical society: Series B (statistical methodology)*, 71(2), 319-392.